DOCUMENTACIÓN TÉCNICA

DistroForge

Manual completo: arquitectura de generadores, guía de los seis pasos, CI generado, modo advanced con ISO custom y núcleo en Rust.

Versión 0.2.0 · Octubre 2026 ·
github.com/Cisne-047/DistroForge

1 - Que es DistroForge

DistroForge es un constructor web de distribuciones Linux personalizadas: eliges base, escritorio, paquetes, repos y sistema, y la aplicacion genera el proyecto de construccion completo - receta del motor, build.sh local y workflow de GitHub Actions - listo para descargar como zip. Todo corre en el navegador: no hay backend, no hay cuentas, y la configuracion vive en localStorage con autosave.

Filosofia

La config es la fuente de verdad. Cada proyecto generado incluye distroforge.json: puedes reimportarlo en la app, editarlo en git y reconstruir, o compartirlo con un enlace (la config se codifica en base64url en el fragmento # de la URL). El asistente es solo una interfaz sobre ese fichero.

Front-end de motores existentes, no un backend propio. DistroForge delega en KIWI NG, live-build, mkarchiso y kickstart + livemedia-creator, y los empaqueta en CI. Eso elimina la division que mato a SUSE Studio: cualquier push reconstruye la imagen.

Seguridad explicita. Las contrasenas se incrustan en claro en las recetas (asi funcionan los kickstart y presets de todos los motores); la app avisa de ello en el README generado y en la UI.

2 - Arquitectura

La app es una SPA React + TypeScript servida en GitHub Pages (https://cisne-047.github.io/DistroForge/). El nucleo logico es puro y portable; en la version 0.2.0 se duplico en Rust (crate forge-core) con CLI, bindings WASM y scaffold Tauri.

Modulo	Responsabilidad
src/types.ts	Tipos: DistroConfig, ExtraFile, ExtraUser, ServiceToggle, NetworkConfig, GrubConfig, CustomIsoConfig
src/catalog.ts	14 bases (desktops, sugerencias, mirrors, presetRepos), plantillas de escritorio, defaultConfig, mergeConfig, archsForEngine
src/templates.ts	Plantillas rapidas: Servidor minimo, Kiosko, Oficina, WSL dev, Multimedia
src/steps.tsx	UI de los 6 pasos (Base, Sistema, Branding, Repos, Paquetes, Construir) con vista previa en vivo
src/App.tsx	Estado global, autosave, Mis distros, carga por URL hash
src/generators/index.ts	generateProject (router por motor), validate, workflow YAML, README generado, zip
src/generators/{motor}.ts	Un fichero por motor: livebuild, kiwi, archiso, kickstart, dockerfile (OCI), wsl, manual, customiso (advanced)
src/generators/accounts.ts	Usuarios, servicios, firewall, autostart, target de arranque, auto-login
src/generators/network.ts	Red estatica: validacion IPv4, mascara dotted, keyfile NetworkManager
rust/ (crates)	forge-core (espejo del nucleo), forge-cli (escritorio/CLI), forge-wasm (web), tauri-app (desktop)

Flujo de datos: la UI muta un unico objeto DistroConfig; generateProject(cfg) lo compila en un mapa ruta-contenido (el zip); validate(cfg) devuelve advertencias accionables que la UI muestra en el paso 6. No hay efectos ocultos: el generador es una funcion pura, lo que permite tests exhaustivos y el port a Rust.

3 - Guia de usuario: los seis pasos

1 - Base. Plantillas rapidas (un clic aplica un preset completo), 14 bases, escritorio (hasta 10 por familia), arquitectura (x86_64, aarch64, armhf, i386 - segun motor) y modo advanced con ISO custom. Una mini vista previa muestra base, motor y escritorio elegidos.

2 - Sistema. Hostname, usuario principal con contrasena, zona horaria, locale, teclado, red (DHCP o IP estatica con prefijo CIDR, gateway y DNS), usuarios extra con rol admin, servicios systemd a habilitar/deshabilitar, firewall con puertos abiertos, target de arranque (auto/grafico/texto) y autostart de programas.

3 - Branding. MOTD, wallpaper (aplicado de verdad via dconf/xfconf), opciones GRUB (timeout y parametros del kernel) y auto-login - todo con vista previa WYSIWYG: un mock de GRUB y un mock de escritorio reflejan la config al instante.

4 - Repos. Repositorios custom con alias, URL, suite/comps y clave GPG. La key se importa de verdad: signed-by en apt, rpm --import en zypper y en el %post de kickstart.

5 - Paquetes. Alta libre con Enter, sugerencias por familia y apps Flatpak de Flathub (chips con borrado).

6 - Construir. Formato de salida (7 opciones), visor de la receta generada fichero a fichero, validacion con advertencias, descargar zip, guardar/cargar config JSON, copiar enlace para compartir y comando QEMU de Test Drive local.

4 - Bases, motores y arquitecturas

Base	Motor	Arquitecturas	Salidas
openSUSE Tumbleweed / Leap 15.6	KIWI NG	x86_64, aarch64	ISO, QCOW2, USB/raw, VMDK, VDI, OCI, WSL
Debian 12, Ubuntu 24.04, Mint 22, elementary 8, Pop!_OS 22.04, Zorin 18	live-build	x86_64, aarch64, armhf, i386	ISO, OCI, WSL
Arch, Manjaro, CachyOS	mkarchiso	x86_64	ISO, OCI, WSL
Fedora 44	kickstart + livemedia-creator	x86_64, aarch64	ISO, OCI, WSL
Gentoo, Solus	manual (guia)	x86_64	Lista de paquetes y guia
Cualquiera (modo advanced)	remaster xorriso	la de la ISO aportada	ISO

El motor se deduce de la base; la arquitectura se limita a lo que cada motor construye de forma fiable (validate() lo avisa). Para CI multi-arquitectura, el workflow generado usa runners nativos ubuntu-24.04-arm para aarch64 y docker/setup-qemu-action para armhf.

5 - Formatos de salida

Salida	Que genera	CI
ISO	Live/instalador hibrido del motor	contenedor del motor; artefacto out/*.iso
QCOW2 / USB / VMDK / VDI	Disco VM o raw via KIWI (image=qcow2/oem/vmx)	mismo job KIWI; out/*
OCI	Dockerfile real + docker save; sin --privileged	docker build + SBOM (syft) + escaneo CVE (trivy)
WSL	rootfs tar con /etc/wsl.conf (usuario, systemd)	contenedor por familia; wsl --import en Windows

Salida	Que genera	CI
ISO (advanced)	Remaster de tu ISO base con xorriso	runner ubuntu + sudo + xorriso/squashfs

6 - Funciones avanzadas y su cobertura por motor

El nucleo aplica cada feature con el mecanismo nativo del motor. La tabla resume la cobertura; validate() avisa de los huecos conscientes (por ejemplo, red estatica en la ISO de Arch, o lo que OCI/WSL ignoran por diseno).

Feature	live-build	KIWI	archiso	kickstart	OCI	WSL
Usuarios extra + admin	hook chroot	XML users	unidad oneshot	lineas user	RUN useradd	chroot
Servicios systemd	hook	config.sh	oneshot enable --now	%post	-	chroot
Red estatica	keyfile NM + chmod	keyfile NM	aviso (NM DHCP)	network --bootproto=static	-	-
GRUB timeout/params	--grub-timeout + bootappend	kernelcmdline + bootloader	sed en build	bootloader --timeout/--append	-	-
Firewall + puertos	firewalld + firewall-cmd	igual	igual (oneshot)	firewall --service/--port	-	-
Flatpak	hook 0350	config.sh	servicio 1er arranque	%post	RUN	chroot
Ficheros extra	includes.chroot	root overlay	airootfs	%post heredoc	COPY files/	heredoc
Auto-login	SDDM/GDM/LightDM	igual	igual	igual	aviso	aviso
Autostart	.desktop en ~/.config/autostart	igual	igual	%post + chown	aviso	aviso
cloud-init NoCloud	includes + paquete	config.sh	airootfs	%post	aviso	aviso
Wallpaper	dconf/xfconf + hooks	root + dconf update	servicio dconf boot	aviso (ignorado)	aviso	aviso

7 - CI/CD generado

Cada zip incluye .github/workflows/build.yml: sube el proyecto a un repo de GitHub y el artefacto de Actions es tu imagen. El YAML se valida en los tests de la app (parseo de todas las combinaciones base por salida). Extras: SBOM SPDX con anchore/sbom-action y escaneo HIGH/CRITICAL con trivy en la salida OCI; runners ARM nativos y QEMU para cross-compile; distroforge.json dentro del proyecto para round-trip y diffs en git.

El repo de la app tiene su propio CI (npm ci + tests + build en cada push/PR) y deploy automatico a GitHub Pages (workflow deploy.yml).

8 - Modo advanced: ISO custom

Para bases exoticas sin builder publico: marcas Usar una ISO custom como base, aportas la URL del .iso y opcionalmente un script chroot propio. DistroForge genera advanced/download.sh, advanced/chroot.sh (usuarios, MOTD, ficheros, servicios, GRUB y tu script) y advanced/remaster.sh: extrae la ISO con xorriso -osirrox, localiza el squashfs raiz, aplica tu config en chroot, reconstruye el squashfs y repack con los parametros de arranque originales (xorriso -report_el_torito as_mkisofs). Es experimental: requiere root y xorriso/squashfs-tools.

9 - Nucleo Rust y versiones de la app

La v0.2.0 porta el nucleo a Rust bajo rust/ con tres crates y un scaffold de escritorio:

Crate	Papel
forge-core	Espejo idiomatico y documentado del generador TS: mismos tipos (serde), catalogo, generadores, validate y workflows; bateria de tests propia
forge-cli	Version de escritorio/CLI: forge dump base dir [salida] [config.json] - vuelca proyectos sin abrir el navegador
forge-wasm	Bindings wasm-bindgen: generate_project / validate consumibles desde la web (target wasm32-unknown-unknown)
tauri-app	Scaffold Tauri 2 (src-tauri) que envuelve la UI y expone el core como comando nativo - binario de escritorio

La version web es la app React desplegada en Pages; la de escritorio, el binario Tauri (o el CLI). Ambas comparten el mismo core de Rust y el mismo distroforge.json.

10 - Desarrollo y calidad

Tests: 85 tests (vitest) que cubren presencia de features por motor, validaciones, YAML de CI de todas las combinaciones y un QA de sintaxis que pasa bash -n a todos los scripts generados. El crate Rust tiene su propia bateria (cargo test).

Inspeccion de recetas: npx vite-node scripts/dump.ts [base] [dir] [salida] [config.json] vuelca a disco lo que generaria la app - asi se audita una receta sin el navegador.

Calidad: el codigo paso dos auditorias automaticas (revision de codigo y build de humo de las 40+ combinaciones en un clon limpio) cuyos hallazgos - escapes de contrasenas, grupos admin por familia, sudoers en OCI, repos ignorados en ramas kiwi/fedora - estan corregidos.

11 - Seguridad

Las contrasenas viajan en claro en las recetas (limitacion estructural de kickstart/presets) y en los enlaces para compartir: no publiques repos de proyectos generados con tu contrasena real; los tests solo usan credenciales de prueba. La app lo avisa en el README generado, en la UI y junto al boton de compartir.

12 - Roadmap

Paridad restante con SUSE Studio: logo personalizado (Personalize), EULA de primer arranque, ISO de instalacion desatendida (preseed/AutoYaST), estimacion de tamano de imagen, Test Drive VNC real en navegador (requiere backend) y galeria publica (requiere backend). Post-paridad: firmado de imagenes, resolucion de dependencias de paquetes y salidas de nube (AMI/VHD).